

Quantum-Enhanced Optimization of Routing Problems on Weighted Graphs

Jørgen Vestly

Department of Physics and Center for Computing in Science Education,
University of Oslo, N-0316 Oslo,
Norway

Route-finding applications are ubiquitous in an interconnected society, with applications ranging from logistics to telecommunication. The *traveling salesman problem* (TSP) is a known instance of a weighted graph problem, belonging to the NP-hard class. This is the class of problems, which has no known practical solution, making it computationally infeasible. This research considers the potential for the QAOA (*Quantum Approximate Optimization Algorithm*) to scale for large search spaces, and if quantum optimization can serve as an enhancement for the classical TSP optimization methods. The results suggests that QAOA is strictly better than the classical optimizers *Greedy search* and *2-opt* for five cities on classical simulation, and six cities on a quantum computer. However, limitations such as noisy computers and limited classical computing power, introduces difficulties for QAOA to scale. We also consider how different state initializations and configurations can affect the way QAOA is optimizing. The paper also discusses the implications of the results and future outlook. NISQ-era hardware makes it hard to demonstrate any significant improvement over classical methods, but promising future technologies might elevate this field to a viable direction.

CONTENTS

I. Introduction	1
II. Theory and methods	2
A. Preliminaries	2
B. Computational complexity and Turing Machines	3
C. Quantum optimization	4
III. Implementation	5
1. Data and setup	5
2. Experimental design	7
3. AI tools	7
IV. Results and discussion	7
1. Classical simulation	7
2. Running on real quantum hardware	8
3. Expanding the classical tests	8
V. Conclusion and future trajectories	10
References	11
A. Derivations of Decomposition of Cost and Mixer Unitary	11
1. Euler Formula	11
2. Derivation of Decomposition of the Mixer Unitary	11
3. Derivation of Decomposition of the Cost Unitary	12

I. INTRODUCTION

Complexity is an intrinsic feature of the universe, and navigating it has perplexed mankind for a long time. Formalizing complexity, and making sense of big data, has been an active pursuit for all kinds of scientists for hundreds of years. Interestingly, there are still many problems which does not have an efficient solution, due to a lack of a known *polynomial time* algorithm. This chasm defines a broad class of problems, known as the NP-class. However, such problems can be optimized to a certain error margin by optimization algorithms. The

Traveling Salesman Problem, abbreviated TSP, is known to be one of the hardest problem in NP, and therefore belongs to the class NP-hard. Formally, the objective of TSP is to return the shortest route on a graph through all vertices, where each vertex can only be visited once. Though the objective is clear, the complexity of possible routes is factorial with respect to n , i.e. TSP has time-complexity $O(n!)$. The traveling salesman problem appears wherever the order of decisions matters. As described in (Matai *et al.*, 2010), TSP-like structures occur in applications such as printed circuit board drilling, warehouse order picking, vehicle routing, scheduling, computer wiring, and X-ray crystallography. In these settings, the “salesman” may be a delivery vehicle, a warehouse robot, a manufacturing tool, or even an abstract scientific measurement process. This makes TSP more than a routing puzzle: it is a compact model of how small ordering choices can accumulate into large real-world costs. The best algorithm for solving TSP is the *Held-Karp* algorithm with $O(n^2 2^n)$, which is exponential. However, in many applications, an approximation to the solution is satisfactory. Therefore, we will investigate approximate methods with the use of the quantum approximation algorithm, QAOA, to find an optimal TSP route in a graph.

The report is divided into *Theory and Methods* II, which introduces the necessary theory behind QAOA and graph problems, to conduct the numerical experiments. Graph theory, computational complexity and quantum optimization is emphasized in this section. The section on *Implementation* (III) explains how the theory is implemented as workable code, and assumptions and limitations are discussed. The *Results and discussion* part (IV) displays and discusses the results obtained

from experimentation. It focuses on performance analysis and the discussion on shortcomings and limitations of the problem. We include tests for both classical simulation and real quantum hardware. The QAOA struggles to simulate more than five TSP cities, despite using different parameters and versions of the algorithm. QAOA on TSP remains an interesting puzzle, but a practically hard problem. The final section, *Conclusion* (V), ties together the most important results from the research, and discusses their implications. We suggest that quantum hardware is still very limited to truly exhibit any advantages for NP-complete problems, and that future technologies might have the capacity to provide an enhancement in performance and speed for such problems.

II. THEORY AND METHODS

A. Preliminaries

This paper assumes that the reader has basic knowledge of quantum mechanics and linear algebra, such as Hilbert spaces, time evolution, Pauli operators, projectors, measurements and quantum gates. In this subsection, some fundamentals will be introduced. Among them are optimization, machine learning, and basic graph theory.

Some computational problems are practically unsolvable by classical computers, and succumbing to *optimization techniques* is therefore the current practice. Optimization methods do not provide the exact solution to a problem, but approximates it with a given error rate, ϵ . In statistics and machine learning, this is done by using a *loss function*, $L(\theta)$. There are several types of loss functions, depending on the task at hand, but the goal is to minimize this to attain the least amount of error from the actual analytical solution (Goodfellow *et al.*, 2016). Before doing any optimization, we must define the problem. We will introduce the *weighted graph* data-structure, and specifically the *traveling salesman problem*.

A *graph*, G is defined by the parameters $G = (V, E, \Phi)$, where V and E denotes two finite sets of *vertices* and *edges*, respectively (Bierlaire, 2018). Vertices are the nodes and edges are the connections between nodes. One can think of a node as a train station, and an edge connecting that train station to another. The *incidence function*, Φ , maps an edge to a vertex-pair, $\Phi_i(e_i) = \{V_i, V_j\}$. A *weighted graph* has a weight associated to every edge, analogous to the cost of traversing the edge. Figure 1 illustrates a simple weighted graph. Note that the edges can be traversed in both directions and the graph is *undirected*. One can see that there are many ways of going from vertex 2 to vertex 5, but that

the cheapest way is the sequence $V_2 \rightarrow V_1 \rightarrow V_5$, with a cost of $7 + 5 = 12$. What if the objective is to visit all the vertices, starting from V_i and returning to V_i , with as the lowest possible cost? This problem is called the *traveling salesman problem*. Imagine a salesman who wants to visit a given set of cities on a map, but since he is notoriously busy, he wants to minimize the amount of time spent to visit all cities. It is important to emphasize the explosive nature of TSP (see Table I). Given n cities, we have $n!$ possible routes (permutations), making the problem practically unsolvable for large instances. How would we go about this problem?

Firstly, we must check if the graph is traversable based

Number of Cities (n)	Unique Routes (n)!
5	120
10	3 628 800
20	2.4×10^{18}
50	3.01×10^{64}

TABLE I Growth of Unique TSP Routes for different values of n

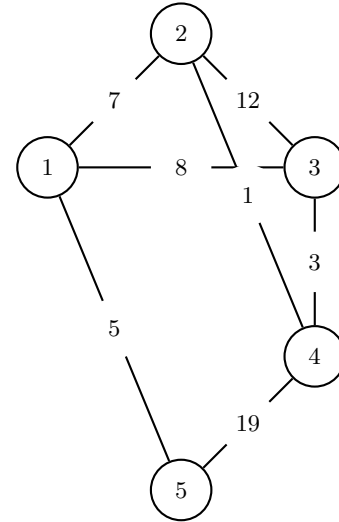


FIG. 1 A weighted graph with 5 nodes. The index inside each vertex points to the vertex number, and the weight is specified along each edge.

on the constraints we have. This is reduced to finding a *Hamiltonian cycle* in the graph, meaning that every node is visited exactly once. Suppose that the traveling is only done by flight, and one can imagine that the salesman has to visit an airport more than once to get to all cities. Hence, the graph has no Hamiltonian cycles, and the TSP will remain unsolved for the specific constraints. Although this paints a general picture, we often assume that the graph is traversable, i.e. having a Hamiltonian cycle, and the objective would now be to implement an algorithm finding the cheapest path. The naive approach would be to consider all possible routes,

and returning the one obtaining the least cost. Suppose we have a route, x , being a list of n ordered cities, we can formulate the cost function for TSP as

$$C_d(x) = \sum_{0 < i, j < n} w_{ij} \sum_{t=0}^{n-1} x_{i,t} x_{j,t+1}, \quad (1)$$

where w_{ij} denotes the distance from city i to city j . An improvement of the naive algorithm is the *greedy search*, which visits the closest vertex at each time step. Although, it likely won't return the fastest route, it is definitely an improvement from the naive algorithm, since the greedy search is *informed*, meaning that the algorithm embeds some knowledge about the problem into its search. A solution can be represented as an ordered permutation, x , where $x_i \in \mathbb{N}$ maps to a fixed city. In turn, we can map this permutation to a matrix to keep track of the two constraints: (i) a city can only be visited once, and (ii) the salesman can only travel to one city at a time. We write $x_{i,t}$, where $x_{i,t} = 1$ means that city i is visited at time, t . Machine learning techniques serves as a further powerful speedup, implementing a direct way of minimizing the cost function. We will consider this in detail when introducing quantum optimization in IIC.

B. Computational complexity and Turing Machines

What is computable? This is a bold question to ask; given a problem, how do we know whether it can be computed? The british mathematician and computer scientist, Alan Turing, formalized (among others) the field *computability theory* in the 1930s in his paper *On Computable Numbers*. He suggested that from a single problem, the *Halting Problem*, one could determine whether a problem was computable (Turing, 1936). The Halting Problem simply states that given an input, x , and a computer, C - it cannot be determined whether the program will last forever, or halt (Karabeg and Djurhus, 1999). The problem is called *undecidable* since it cannot be decided by a *Turing machine*. Moreover, by knowing that the Halting Problem is undecidable, we can also prove that other problems are undecidable by so-called *reductions*.

A Turing Machine is an mathematical model of computation, that can simulate any computer program (Mullins, 2012). For decision problems, it can for any input (binary instruction set), x , map to a yes-or no instance. For the Halting problem, the Turing machine is incapable of returning a binary decision instance for any input, x . But suppose we were to do some operations in a finite loop, then the Halting machine would halt on the input, and produce a 'yes'. In other words, it is a universal machine asserting that anything which is computable, is computable by this machine. The latter

statement is commonly referred to as the *Church-Turing thesis*. If this thesis is universally applicable, it means that, in extension, a Turing machine can handle any quantum computation. So in theory, quantum computers cannot ever solve problems that classical computers can (Larsen, 2024). We have now only considered decidability, meaning that a given problem is solvable, without mentioning *efficiency*. We have established that some problems are simply unsolvable, but among the solvable problems, we need to define *complexity classes* to determine their efficiency.

We define the complexity class, $\mathbb{P}$, to contain all problems that are solvable in polynomial time, meaning that they belong to $O(n^k)$. These problems are, by the standard of modern computers, efficiently solvable. Formally, we say that a Turing Machine halts after atmost n^k steps, i.e. in polynomial time. A problem belongs to the NP class, if it is solvable in *non-deterministic* polynomial time. This means that the problem can be solved in polynomial time by a *non-deterministic* Turing machine (see Figure II.B), being a machine of branched computation where the program might halt, or loop forever. The machine computes in parallel, being somewhat reminiscent to the computational model of a quantum computer, where we add a probabilistic component where each edge has an assigned probability, $|\Psi|^2$, making it a *probabilistic* Turing machine (of Cambridge, 2019). The theoretical capacity for *quantum parallelism* is a big motivation for building large-scale quantum computers. We define the *bounded-error quantum polynomial time* (BQP) as the class of problems, where given a decidable instance $A = (A_{yes}, A_{no})$, there exists a polynomial-time generated family of circuits, $Q : \{Q_n : n \in \mathbb{N}\}$, and each circuit Q_n takes n -input qubits and produces and output satisfying

$$(i) \text{ if } x \in A_{yes}, Pr[Q \text{ halts on } x] \geq p_y, \quad (2)$$

$$(ii) \text{ if } x \in A_{no}, Pr[Q \text{ halts on } x] \leq p_n, \quad (3)$$

where we define $\varepsilon = (\epsilon_y, \epsilon_n) = (2/3, 1/3)$ to be the error-bounds of the algorithm (Watrous, 2008). The program halts on input, x , with a probability of 2/3 for the 'yes'-instances, and with a probability of 1/3 on the 'no' instances. The BQP-class can be thought of as all problems solvable by nature as it is, by means of quantum mechanics. The enormous interest for showing that $\mathbb{P} = \text{NP}$, could in theory be shown indirectly, by proving that $\text{BQP} \subseteq \text{NP}$. This would mean that quantum computers could in principle solve all problems in NP (Watrous, 2008).

Problems in NP are particularly interesting, since their solutions are hard to obtain, but once given, easy to verify (Karabeg and Djurhus, 1999). The problems in NP does not have any known polynomial-time solution,

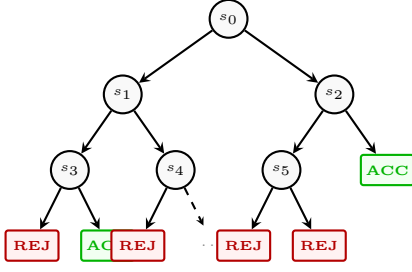


FIG. 2 A tree illustrating the decisions of a Nondeterministic Turing Machine (NTM)

but if a solution is given, then it can be verified in polynomial time. TSP is in NP , since given a tour, it is easy to verify that each vertex is visited exactly once. Also, as discussed earlier, there are no polynomial-time algorithms solving TSP in the general case. Problems in NP that are at least as *hard* as any other problem in NP are *NP-complete*. *Hardness* for a given problem, L , is achieved if all problems in a given class is *polynomial-time reducible* to L . A reduction is a way of proving that a problem B is at least as hard as a known problem A . We know that the Halting problem, L_H is NP-hard (Karabeg and Djurhus, 1999), and this can be reduced to the Hamiltonian cycle problem. We will not be stating the proof here, but this is a known reduction (Karabeg and Djurhus, 1999), and it proves that the Hamiltonian cycle problem is also NP-hard . A reduction is simply a way of transforming a problem into another, and it can be proved that Hamiltonian cycle can be reduced to TSP. Hence, TSP is NP-hard , and since it is in NP , also NP-complete .

When it comes to complexity theory, there is no proof that quantum computers can solve problems that classical machines cannot. However, the exciting part concerns the temporal aspect - even though classical problems in NP might be solvable, it may require too much time for it to be solved in practice. Grover's algorithm (1996) remains one of the most interesting speedup algorithm for quantum computers. The algorithm is for searching in an unsorted database, and has therefore wide applicability. Showing a time complexity of $O(\sqrt{n})$ vs. $O(n)$ for classical search. In the following subsection, we will consider quantum optimization, as a possible enhancement of classical, for the TSP problem.

C. Quantum optimization

We have considered problems with *exact* solutions when discussing complexity theory. However, in the real world, one often acknowledge that decisions are based on the best possible solutions, *approximations*. The

fact that $\text{TSP} \in \text{NP}$, does not make it impossible to find nearly ideal routes for the salesman, let alone very good ones, even for large number of cities. We introduce QAOA (*Quantum Approximation Optimization Algorithm*) to tackle classically intractable combinatorial problems, with applications for TSP specifically. QAOA is a *hybrid algorithm*, using a classical optimizer but a quantum configuration (state space) (Qian et al., 2023). The main idea of the algorithm is to use a Hamiltonian, $\hat{H}_C$ as a cost function, being subject to minimization. The QAOA algorithm consists of three parts. The first one is **state initialization**, which is done by superposing all qubits using the Hadamard gate, H :

$$|+\rangle^{\otimes n} = H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{2^{\frac{n}{2}}} \sum_{x \in \{0,1\}^n} |x\rangle,$$

creating a state space of 2^N amplitudes. There are other ways of initializing the state space (Hjorth-Jensen, 2026). For TSP we can denote a city as $x_{i,t} = (\mathbb{I}_{i,t} - \mathbb{Z}_{i,t})/2$, mapping a binary state space to a quantum state space: $\{0,1\}^n \in \mathbb{B} \rightarrow |\Psi\rangle \in \mathbb{H}$. We define that $x_{i,j} = 1$ corresponds to the city i being visited at time t , and we can assign each city to a qubit. The next step is to define the **unitary evolution**. We define the *unitary evolution operator* for an initialized state $|s\rangle$ as

$$\begin{aligned} \hat{U}(\gamma, \beta) |s\rangle &= \prod_{i=1}^n e^{-i\gamma_i \hat{H}_C} \prod_{i=1}^n e^{-i\beta_i \hat{H}_M} |s\rangle \\ &= \hat{U}_C(\gamma) \hat{U}_M(\beta) |s\rangle, \end{aligned}$$

where a ground state for $\hat{H}_C$ is aimed to be found. $\hat{H}_M$ is called the *mixer Hamiltonian*. This Hamiltonian does not commute with the cost Hamiltonian, $\hat{H}_C$, and it explores the solution space by switching between the basis states, $\hat{H}_M = \sum_{i=1}^n \mathbb{X}_i$. The QAOA algorithm can be implemented as a parametrized circuit with n layers (see Figure 3). We can decompose the circuit blocks into unitary quantum gates (Hjorth-Jensen, 2026). $\hat{U}_M(\gamma_i)$ can be decomposed as

$$\hat{U}_M(\gamma_i) = \bigotimes_{i=1}^n e^{-i\beta X_i} = \bigotimes_{i=1}^n R_x(2\beta),$$

using the idempotent element $X_i^2 = I$, and the Euler Formula (see Appendix). This leaves us with a set of X-rotation gates, where $\hat{U}_M$ rotates qubits around the x-axis. $\hat{U}_C$ can be decomposed as

$$\hat{U}_C(\gamma_p) = \prod_{\substack{i,j=1 \\ i < j}}^n \text{CNOT}(i, j) [\mathbb{I} \otimes R_z(2\gamma_{ij}^p)] \text{CNOT}(i, j) \prod_{i=1}^n R_z^i(2\gamma_i^p).$$

The derivation is quite complex, and given in the Appendix. This brings us one step closer to implement the unitary blocks as parametrized circuits. The cost Hamil-

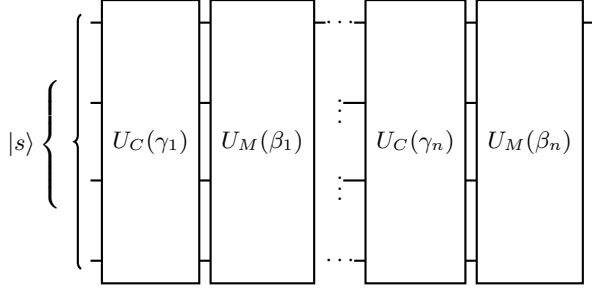


FIG. 3 QAOA Circuit with n layers.

tonian is the *qubitized* cost function, mapping the classical cost function 1, into Hilbert space. Note that the cost function does not prohibit a city being visited more than once, so an additional *penalty term*, C_p , might be added for assuring that the route corresponds to a Hamiltonian cycle. This is achieved when $\sum_{t=0}^{n-1} \sum_{i=0}^{n-1} x_{i,t} = 1$ for all $i = 0, 1, \dots, n-1$. We can penalize this with the squared L2-norm, given by

$$C_p(x) = \lambda \left\{ \sum_{t=0}^{n-1} \left(1 - \sum_{i=0}^{n-1} x_{i,t} \right)^2 + \sum_{i=0}^{n-1} \left(1 - \sum_{t=0}^{n-1} x_{i,t} \right)^2 \right\}.$$

Given $C(x) = C_d(x) + C_p(x)$, the cost Hamiltonian can, by the Ising formulation, be expressed as

$$\hat{H}_C = \sum_i c_i Z_i + \sum_{ij} c_{ij} Z_i Z_j.$$

The XY mixer Hamiltonian, $\hat{H}_{M_{XY}}$, is defined as

$$\hat{H}_{M_{XY}} = \sum_{i=0}^{n-1} \sum_{t=0}^{n-1} XY_{(i,t),(i,t+1)}$$

, coming it handy for the TSP problem, since it is constrained (Qian *et al.*, 2023). The standard mixture is the X mixer, using only X-gates, and then also the Grover mixture, looking at global structure by considering the mean of amplitudes, potentially escaping local structure (Hjorth-Jensen, 2026).

The final step is **measurement and optimization**. We have defined the setup for QAOA, but need some meaningful way of processing and reading out the data. QAOA is trained layerwise by using the COBYLA optimizer. COBYLA is a gradient-free optimizer for constrained problems (IBM Quantum, 2026). The optimization terminates when convergence is successfully met. It is common practice to measure the unitary operator on the final layer, $\hat{U}(\gamma, \beta)$ for every iteration through the network, many times, by taking *shots*. From the repeated measurements, we obtain an average bitstring from the qubits after the p -th layer, and this string corresponds to the solution to the TSP, however

we decide to encode it. We verify the classical cost of the TSP solution, and check whether it is an improved solution. Then, we feed the energy expectation value to COBYLA, given by

$$E(\gamma, \beta) = \langle \psi(\gamma, \beta) | \hat{H}_C | \psi(\gamma, \beta) \rangle,$$

where COBYLA in turn returns $\{\gamma', \beta'\}$ which, based on inference, guides the new solution in the direction of lower energy. One can think of COBYLA setting up a proximal gradient, based on the former parameters. We determine a radius, ρ_k , and when the actual shrinkage of the radius, ρ_s , falls below ρ_k , the algorithm concludes that the TSP route is optimized. Since we do not know the actual solution to TSP, we do not have an analytical lowest cost number, and therefore we reduce the problem to shrink some solution radius around the optimal route.

We can think of mapping each qubit to every edge, and then consider the worst case to be a complete graph, where all cities has an edge to every other city except itself: $n(n-1) \in O(n^2)$. When superposing all edges, we get 2^{n^2} amplitudes, being the state space. In principle, the algorithm can process and act on all these amplitudes, but computes projects the highest probability onto a binary state. In principle, the quantum computation can act on the amplitudes simultaneously, without storing the quantum states, potentially allowing for speedup and less memory than classical computation (Markidis, 2024). For combinatorial and NP-complete problems, this very possibility is exciting and worth investigating. Equally interesting is the question of whether the theory will hold in practice.

III. IMPLEMENTATION

We have delved into a theoretical odyssey about complexity, and how combinatorial problems might be approached by quantum computing. Although the theory seems exciting and promising, we must build the bridge from theoretical wonder to practical implementation. How do we actually set up a workable machinery to tackle the TSP problem with quantum optimization? What limitations are there in practice, and what assumptions must be made? This section explains the reasoning behind implementing the executable algorithms, with examples.

1. Data and setup

The codebase for the project (Vestly, 2026) is comprised of Jupyter test files, and *utilities.py* containing the classes and functions needed to perform the testing

and experimentation. We have one class for TSP and one for QAOA. The TSP class contains the methods for doing classical TSP. These methods include Greedy Search (see II), the *2-opt* algorithm, and the *Held-Karp* algorithm. The specifics of the two latter algorithms is beyond the scope of this research, but 2-opt returns an approximate solution, whereas Held-Karp returns the shortest route in $\in O(n^2 2^n)$. These classical algorithms serves as temporal- and performance benchmarks for QAOA. The implementation is taken from (Di Caro, 2026) and (Nguyen, 2020), respectively.

The TSP graph is encoded as a dictionary, where cities constitute the keys, and the values are xy-coordinates, i.e. tuples of $(x, y), x, y \in [0, 100]$. The cities are simply integers, but can be mapped to city names, like (Oslo: 34.76, 61.09).

```
class TSP:
    def __init__(self, n_locations=10):
        self.n_locations = n_locations
        self.map = {}
        self.distance_matrix = []

    def create_map(self):
        self.map = {i: (x_i, y_i) for i in
            range(self.n_locations)}
        return self.map

    def compute_distance_matrix(self):
        self.distance_matrix = [
            [euclidean_distance(i, j) for j
             in range(self.n_locations)]
            for i in range(self.n_locations)
        ]
        return self.distance_matrix

    def is_valid_tour(self, tour):
        return (
            len(tour) == self.n_locations +
            1
            and tour[0] == tour[-1]
            and sorted(tour[:-1]) == list(
                range(self.n_locations))
        )

    def tour_cost(self, tour):
        return sum(
            self.distance_matrix[tour[i]][
                tour[i + 1]]
            for i in range(len(tour) - 1)
        )

    def random_tour(self, start=0):
        cities = list(range(self.n_locations))
        cities.remove(start)
        random.shuffle(cities)

        return [start] + cities + [start]
```

```
def greedy_search(self, start=0):
    return greedy_tour

def two_opt(self, initial_tour):
    return improved_tour, improved_cost

def held_karp(self, start=0):
    return optimal_tour, optimal_cost
```

Listing 1 Minimal classical TSP class.

Listing 1 displays a minimal and simplified version of the TSP class. Based on the dictionary, we use Euclidian distance between every point and obtain a *distance matrix*, given by

$$D = \begin{array}{c|ccc} & A & B & C \\ \hline A & 0.00 & 4.72 & 8.15 \\ B & 4.72 & 0.00 & 6.38 \\ C & 8.15 & 6.38 & 0.00 \end{array}, \quad (4)$$

where A,B,C corresponds to the cities, and where e.g. $D_{1,2}$ denotes the distance from A to B. The diagonals are zero, since the distance from a city to itself must be zero. The matrix is symmetric, since the distance $d(A, B) = d(B, A)$. The method *is_valid_tour()* checks if a given tour is a Hamiltonian cycle, i.e. a valid encoding, asserting that the first and last city is equal, and that no cities are visited more than once. The method *tour_cost()* computes the total cost (length) of the tour using the distance matrix as a look-up table. *random_tour()* creates a random tour, which is to be optimized by the *two_opt()*. The other methods needs just the start node to build a tour.

Now we will consider the implementation of the QAOA class. We must map the classical states to quantum states, by qubitizing (see Section 2C). Each qubit contains $x_{i,t}$, and for n cities, we have 2^{n^2} amplitudes after superposing (see Section 2C). This is done in the *ansatz()* method. We measure a bitstring corresponding to a one-hot encoding of the tour, obtained from a *permutation matrix*. If we have three cities, then the permutation matrix

$$P_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

corresponds to the route $A \rightarrow C \rightarrow B$. Every entry in the matrix is a $x_{i,t}$, so $x_{3,2}$ means that city C is visited at $t = 2$. The measurement is a bitstring of a columnwise permutation matrix. The bitstring for P_3 is {100001010}. The *processing()* method builds a variational circuit based on an ansatz and parameters coming from the COBYLA optimizer in *optimize()*. See the GitHub folder for further documentation of the class (Vestly, 2026). The processing class contains the unitary evolution of the circuit, and one can define

a given number of layers as an argument. When a QAOA instance, q is created, we can simply call on $q.optimize(epochs)$, and then it returns the route with the lowest cost. For efficiency, we choose the ansatz to be the one-hot encoded version of Greedy Search, and superposing all the classical bits.

2. Experimental design

To truly harness the potential of any algorithm, one must systematically test different configurations to find the best set of parameters through *model selection*. In this spirit, we consider the relationship between the tunable parameters, such as number of layers, state initialization, different mixer Hamiltonians, optimizers, number of shots etc. This testing is done 'behind the curtains', and is not interesting in and of themselves, but included in the Jupyter test files (Vestly, 2026).

3. AI tools

AI is omnipresent in modern development and research. Used correctly, it can significantly cut the time spent on writing code and debugging. Throughout this research, I used AI for testing and debugging, but also to write small snippets of code as long as I already understood what was to be implemented, to save time. An example is to write a cost function for the Hamiltonian. I asked chatGPT to generate this method with correct datatypes, and I verified that it worked correctly. Also, for creating test and plot code for comparing methods, I have used AI tools. What I have not used AI on is to construct large parts of my code, where I lose control of code semantics. A prompt doing this could be: "Write the QAOA class from scratch".

IV. RESULTS AND DISCUSSION

In this section, we present the results for QAOA on the TSP. This will be done through a structured analysis and discussion with a critical emphasis on the basis and limitations for the experimental design. We use classical benchmarks to compare with QAOA on a classical computer, but also quantum backend from IBM. The results primarily focuses on different types of initializations, Hamiltonian mixtures, and other relevant heuristics. By themselves, the tests does not exhibit any time- or performance advantage over the classical ones. Albeit, we are interested in how QAOA is solving the problem, and discuss the current limitations and future prospects for quantum optimization on NP-complete problems.

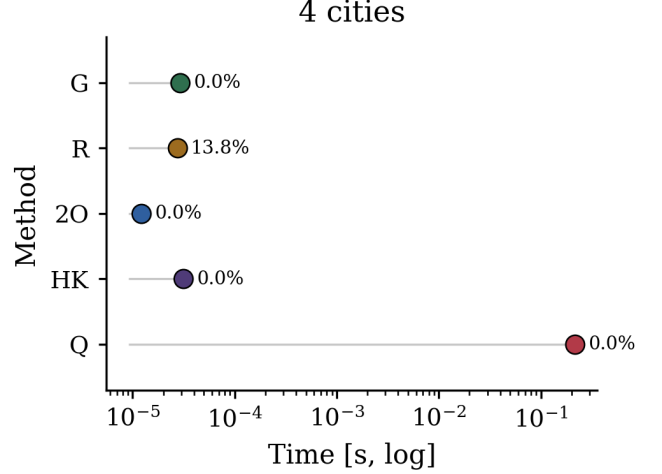


FIG. 4 Comparison in time and performance (cost) for 4 cities for Greedy Search (G), 2-opt (2O), Held-Karp (HK), randomized classical tour (R), and QAOA with greedy initialization (Q). The x-axis shows logarithmic time in seconds, and the percentage on the data points corresponds to the deviation from the actual cost, obtained from Held-Karp. We use 1 unitary layer and 10 shots. The QAOA process is simulated on a classical computer.

1. Classical simulation

When allowing the full search space, 2^n for n cities, the classical simulation of QAOA struggles to compute more than five cities. Figure 4 compares classical methods (Greedy, 2-opt, randomized tour, and Held-Karp) to QAOA with greedy initialized ansatz, in terms of time taken to run the different algorithms and the deviance from the true solution, for four cities. Notice that QAOA finds the fastest route, but spends a lot more time than the classical methods. This is due to the enlarged Hilbert space being processed classically, where the $\dim(\mathcal{H}) = 2^{4^2} = 65536$ states. Since superposition and interference are not classical phenomena, simulating classically will only make the method exponentially slower. The motivation for pruning the search space, i.e. making the Hilbert space smaller, is specifically wanted for classical simulations. Until now, we have included the start node twice, since we include the final node as the start node to finalize the cycle. However, we can ignore the start node as the final node and leave the space to be $(n-1)!$. Also, we know that the cost of (A, B, C) is equal to the cost of (C, B, A) , and we do not have to consider *symmetric routes*, making the new space $\frac{(n-1)!}{2}$ for n cities. So for five cities, we now have $2^{\frac{(5-1)!}{2}} = 2^{12} = 4096$ states, which is significantly smaller than with four cities for the whole space. For five cities

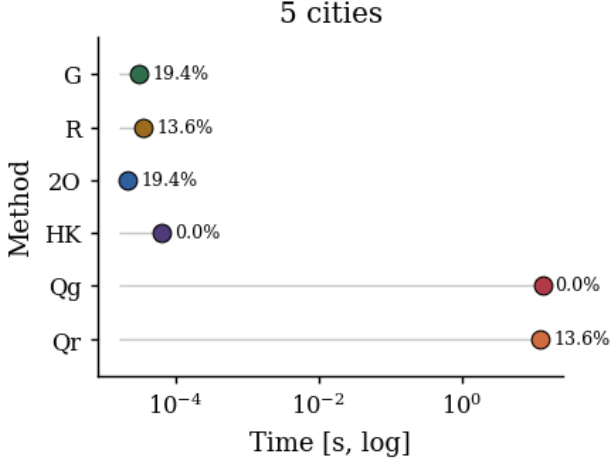


FIG. 5 Comparison similar to Figure 4 but with five cities, and an added Q_r , i.e. a randomly initialized QAOA ansatz, with 1 unitary layer and 10 shots. We used a compressed symmetric space with $\frac{(n-1)!}{2}$ possible routes for n cities. Results from QAOA are simulated classically.

with the pruned space, we see that QAOA with greedy ansatz performs better than the classical optimizers (see Figure 5). Although it spends a larger portion of time, it is interesting that it outperforms 2-opt on a small instance such as five cities. Being fed with an already optimized route from Greedy search might accelerate the optimization further. We see that with a randomly initialized ansatz, the QAOA method performs worse, and does not reach the best route for five cities. This underlines the effectiveness of informed search as a base for quantum optimization.

2. Running on real quantum hardware

We have seen that for five cities, the QAOA finds the shortest route successfully. However, using classical computers, it is infeasible to scale notably over this level. We consider the inclusion of real IBM hardware with 156 qubits, and its way of solving the problem. The caveat of using IBM computers is that we only get an allocated computational time of 10 minutes per month, which heavily suppresses the possibility of comprehensive experimentation. However, we managed to run six cities on real quantum hardware, which was not successfully achieved on a quantum simulator. Figure 6 compares QAOA on a real IBM machine with Greedy, 2-opt, and Held-Karp. We note that, despite the time consumption, that QAOA finds a shorter path than the classical approximators. It used 5 minutes and 46 seconds to converge, but the result is still strictly better than 2-opt. Due to time limits, we used Qiskit's *QAOAAnsatz()*-class, which does not use Greedy as its state initialization, but the standard one explained in II. It would have been

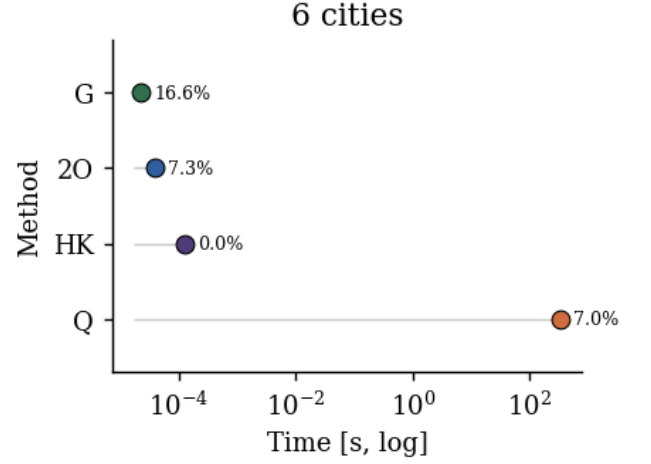


FIG. 6 Comparison of classical optimizers/solvers vs. QAOA with qiskit on a 156 qubit quantum backend, with six cities. The x-axis denotes logarithmic time scale expressed in seconds, and y-axis specifies the methods of use. The plot was generated from using one QAOA iteration and 32 shots.

interesting to consider a Greedy start ansatz to nudge the algorithm to faster convergence, and even lower cost, but the time limits were reached too early. Despite that limitation, it is interesting that QAOA performs better than classical solvers, and can also handle an extra city. To save time spent, we used only one QAOA iteration too, and fewer shots than the standard, and I am sure with more time resources that the method could have scaled to more than six cities.

3. Expanding the classical tests

Thus far, we have only considered cases with a fixed mixer Hamiltonian, number of layers, state initializations etc. We must further explore the effects of tweaking such quantities, and what to learn from the way QAOA is solving the TSP, and not merely consider the performance itself. By experimentation, we found that 2-3 QAOA layers was sufficient for the classical simulation. Since TSP instance of five cities at most, creates small complexity, it is expected that a deeper configuration does not yield better performance. It is also interesting to see how the models inferred the returned route. Since the computation is probabilistic, it returns the most likely answer in terms of the squared amplitudes, $|\alpha|^2$, for valid tours as dots with the magnitude as the size of the dots. These are probabilities for the final layer, i.e. the output of the QAOA. We see that the standard initialization (uppermost) gives a uniform probability distribution, suggesting that the algorithm has not learned to discriminate different tours, giving equal probability of returning each route. For this

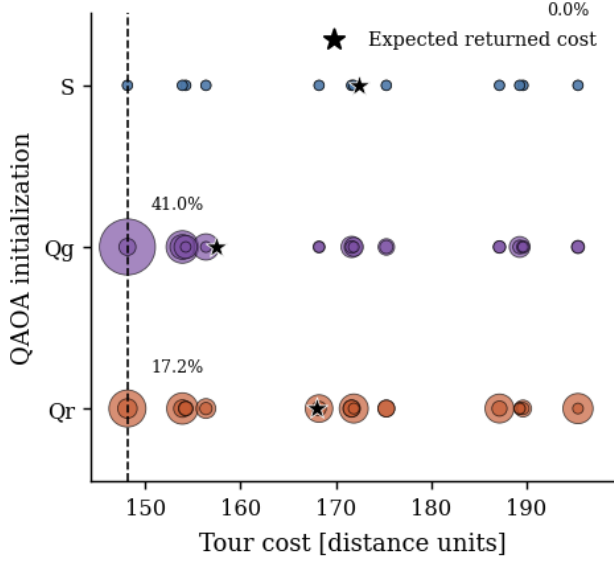


FIG. 7 Probability distribution for returned cost using different state initializations: standard (S) by setting all qubits to $|0\rangle$ and superposing, and then Q_g and Q_r as before. The models computes for five cities with 3 layers, 10 shots, and maximum iterations of 25. The dotted vertical line represents the Held-Karp solution.

specific state initialization, one might need more layers to actually observe improvement. Q_g has its highest probability amplitude on the solution of Held-Karp, with 41%, meaning that it will return the optimal solution 41% of the time. The expected cost, represented as a star, is also very close to the optimal solution. The informed initialization from greedy has arguably established some learning from the start, and then refined the solution through optimization. A randomly initialized state, Q_r , is also very uniform, but some higher probabilities toward the solution. The expected cost is worse than for Q_g , but better than, although closer to, S .

Different mixtures adds different flavours to the computation. As mentioned in the theory section (II), the XY mixer has shown great promise for constrained problems. However, the default X mixer provides the optimal solution for 5-city for the Greedy initialization, despite spending 47s compared to 3.2s for XY (see Figure 8 and 9). Interestingly, XY finds a cheaper tour with the standard initialization, S . Although Grover is praised as an exceptional mixture in the literature, it takes too long for this case, especially for the randomly initialized ansatz. Despite seeing some patterns based on Figures 8 and 9, the real potential for these different mixers may not be expressed, since the instance is too small. For a larger system, one might expect both the XY -and Grover mixers to scale more preferable, and obtain better costs, than the use of X mixer.

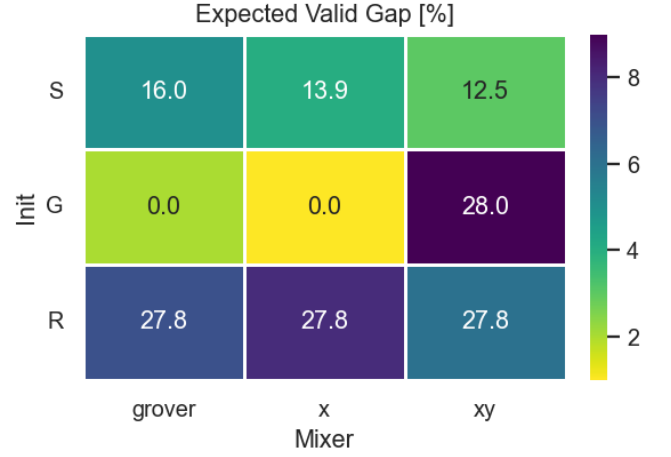


FIG. 8 A heatmap of expected gap from the solution (being 0.0%) for valid tours, with different mixers and state initializations. Using the default 5-city, 3 layer, 10 shots, and 25 max-iters configuration. The units on the colorbar represents some color settings, and does not correspond to percentage.

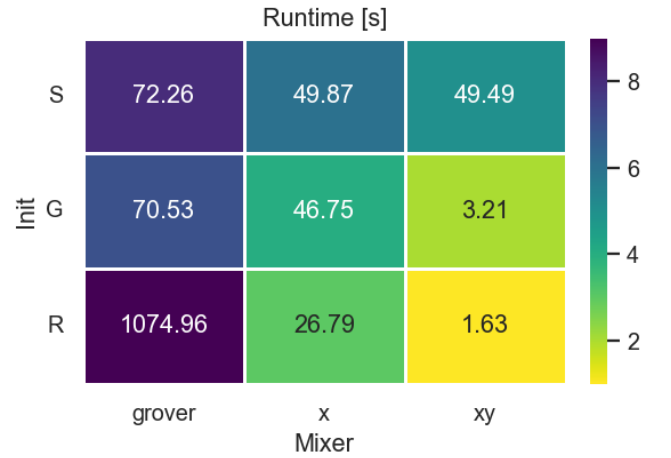


FIG. 9 Heatmap of runtime in seconds for the same models as Figure 8. The units on the colorbar does not correspond to seconds, but some intrinsic color settings with no informational significance.

The plot in Figure 10 shows the convergence of QAOA with depth $p = 3$ using the X mixer for three different initial states. The curves show the best TSP energy found so far as the classical optimizer proceeds. The greedy initialization G and random valid initialization R both reduce the energy rapidly after only a few iterations, while the standard initialization S improves only slightly and remains at a much higher energy. Note that the energy for S should be interpreted differently from the conditional gap shown in the heatmap. The convergence plot uses the full penalized TSP energy, including contributions from invalid one-hot encodings. Since S is a uniform superposition over the full binary

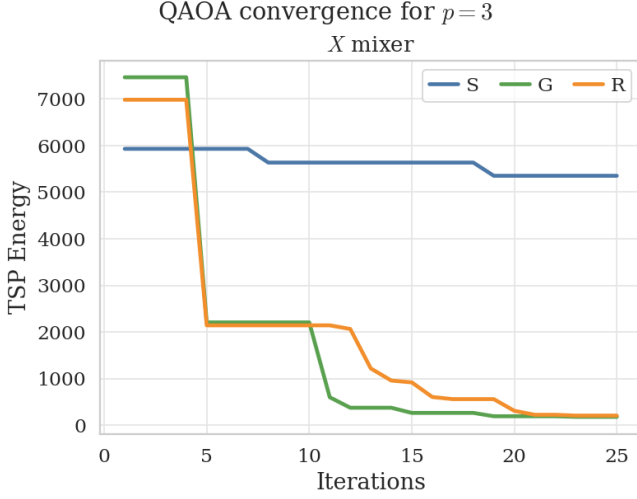


FIG. 10 Convergence comparison between different state initializations, using X mixer. The horizontal axis represents the number of iterations through QAOA, and the vertical axis, the energy for the TSP solutions.

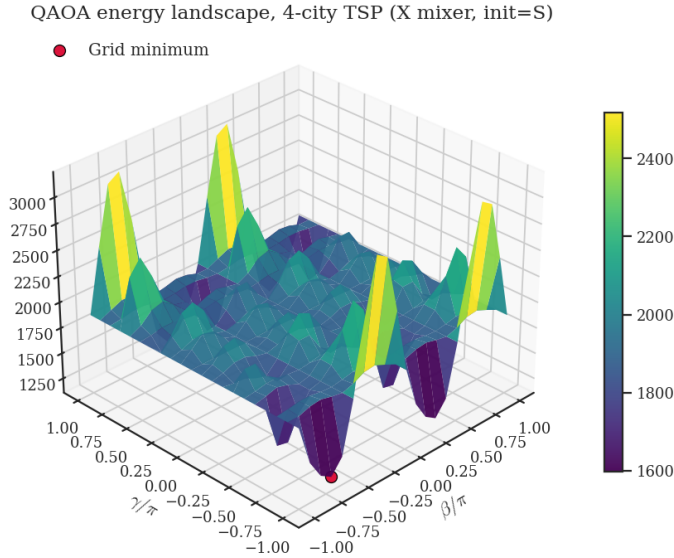


FIG. 11 QAOA energy landscape for the 4-city TSP instance using the X mixer and standard uniform initialization. The surface shows the expectation value $\langle H_C \rangle$ as a function of the variational angles β and γ for $p = 1$. The figure was generated with $n = 4$, $p = 1$, fixed start city 0, seed 10, and a 21×21 grid over $\beta, \gamma \in [-\pi, \pi]$.

space, much of its probability mass lies outside the valid tour subspace and therefore receives large penalties. In the heatmap, however, the expected gap is computed only after conditioning on valid tours, so S can appear more competitive there even when its total probability of producing a valid tour is very small.

Another interesting observation made during ex-

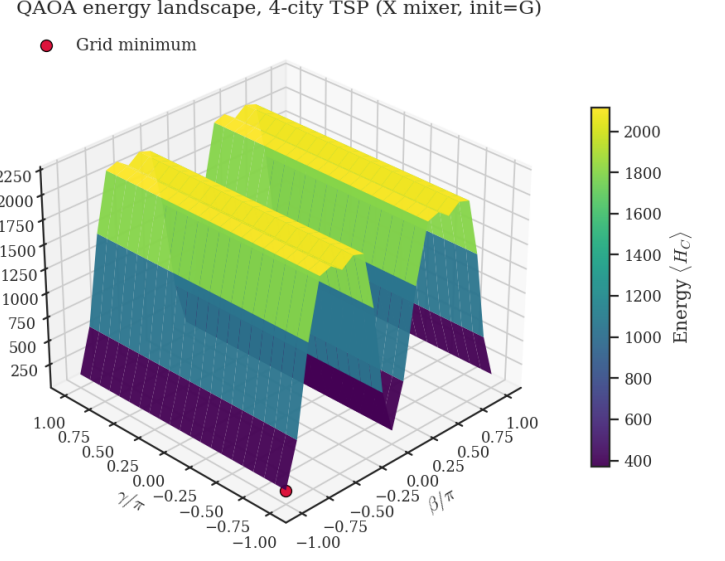


FIG. 12 QAOA energy landscape for the 4-city TSP instance using the X mixer and greedy-tour initialization. The surface shows the expectation value $\langle H_C \rangle$ as a function of the variational angles β and γ for $p = 1$. The figure was generated with $n = 4$, $p = 1$, fixed start city 0, seed 10, and a 21×21 grid over $\beta, \gamma \in [-\pi, \pi]$.

perimentation was the energy landscape created by the optimization for different initializations. In Figures 11 and 12 we see the energies plotted for a continuum of values for $\{\beta, \gamma\} \in [-\pi, \pi]$ for standard initialization and greedy initialization respectively. The red dot is the returned ground state energy. For the standard initialization, we see that the landscape contains smaller and seemingly more random peaks and valleys. At the boundaries, these oscillations has the highest magnitude, but if we look at the energies, the ground state is estimated to be around 1250, as opposed to around 250 for greedy initialization. Furthermore, for greedy, the landscape is more structured, showing a clear parameter dependence, with distinct minima at $\beta, \gamma \in [-\pi, 0, \pi]$. This consolidates our observation of informed learning when using greedy initialization, where the optimization is done smoother and more systematically, compared to the standard initialization, where the learning appears to be random, and not close to converging to the true ground state.

V. CONCLUSION AND FUTURE TRAJECTORIES

Having considered both classical simulation and running on real quantum hardware, with different configurations, the QAOA algorithm seems very promising, albeit on smaller TSP instances. Assuming future noise-free systems, also referred to as the post-NISQ-era technologies, there is no doubt that QAOA can poten-

tially scale and even surpass classical optimizations due to its way of inferring solutions, by considering a more complex space. We showed that with the use of an IBM backend, we could run QAOA on six qubits instead of only five with classical simulation, and it found a route with less cost than the classical optimizers, Greedy search and 2-opt. Although we failed to scale the graph to more than six nodes, it remains an interesting opportunity for using QAOA on noise-free systems. Moreover, due to lack of computing time on the IBM quantum computer, we were not able to test it systematically, and we assume that with free access, these results would have been notably better. However, it is still intriguing to see that the system scales, even on an unoptimized, noisy quantum computer.

Despite there has not been any proof, neither formal nor experimental, that quantum computers can solve NP-complete problems, does the field of quantum optimization remain a rich and potentially transformative endeavor. Many real world problems that are exponential in nature, does not always require the best possible solution, and quantum optimization techniques such as QAOA can potentially scale and speed up such systems. Also, as shown in this paper, despite small instances, it can even produce more optimal solutions than classical optimizers. While mathematicians struggle to tailor quantum complexity classes with classical, the experimental data is also very valuable toward quantum supremacy in some specific domains. For future research, it would be interesting to combine this architecture with some Quantum Error Correction decoding to see if the system can be scaled further.

REFERENCES

- M. Bierlaire (2018), *Optimization: Principles and Algorithms* (EPFL Press, Lausanne).
- U. of Cambridge (2019), “Quantum computing (CST part II) – lecture 12: Quantum complexity,” Department of Computer Science and Technology, lecture Notes.
- G. Di Caro (2026), “[Local search and iterated local search](#),” Lecture slides, Carnegie Mellon University in Qatar, accessed: 2026-04-30.
- I. Goodfellow, Y. Bengio, and A. Courville (2016), *Deep Learning* (MIT Press, Cambridge, MA).
- M. Hjorth-Jensen (2026), “Quantum computing and quantum machine learning: Week 13, qaoa algorithm and implementation,” Lecture notes, GitHub repository, accessed: 2026-05-06.
- IBM Quantum (2026), “COBYLA (v0.46),” <https://quantum.cloud.ibm.com/docs/en/api/qiskit/0.46/qiskit.algorithms.optimizers.COBYLA>, IBM Quantum Documentation. Accessed: 2026-04-27.
- D. Karabeg, and R. Djurhus (1999), *IN210 Kompendium*, Lecture Compendium (Universitetet i Oslo) course compendium for IN210.

- N. S. Larsen (2024), “[Mat3420 lecture notes](#),” Course material, Universitetet i Oslo, lecture notes for MAT3420: Quantum Computing.
- S. Markidis (2024), “[What is quantum parallelism, anyhow?](#)” [arXiv:2405.07222 \[quant-ph\]](#).
- R. Matai, S. P. Singh, and M. L. Mittal (2010), “[Traveling salesman problem: An overview of applications, formulations, and solution approaches](#),” .
- R. Mullins (2012), “[Raspberry pi: Introduction: What is a turing machine?](#)” University of Cambridge.
- Q. N. Nguyen (2020), “[Travelling salesman problem and bellman-held-karp algorithm](#),” Lecture note, accessed: 2026-04-30.
- W. Qian, R. A. M. Basili, M. Eshaghian-Wilner, A. Khokhar, G. Luecke, and J. P. Vary (2023), “Comparative study of variations in quantum approximate optimization algorithms for the traveling salesman problem,” [arXiv:2307.07243 \[quant-ph\]](#).
- A. M. Turing (1936), “On computable numbers, with an application to the entscheidungsproblem,” [Proceedings of the London Mathematical Society](#) **s2-42** (1), 230–265.
- J. Vestly (2026), “Fys5419_p2,” https://github.com/JVestly/FYS5419_P2, gitHub Repository.
- J. Watrous (2008), “Quantum computational complexity,” [arXiv preprint arXiv:0804.3401 10.48550/arXiv.0804.3401](#), [arXiv:0804.3401 \[quant-ph\]](#).

Appendix A: Derivations of Decomposition of Cost and Mixer Unitary

1. Euler Formula

For any real number θ , Euler’s formula is

$$e^{i\theta} = \cos(\theta) + i \sin(\theta).$$

Similarly,

$$e^{-i\theta} = \cos(\theta) - i \sin(\theta).$$

More generally, if $A^2 = I$, then

$$e^{-i\theta A} = \cos(\theta)I - i \sin(\theta)A.$$

2. Derivation of Decomposition of the Mixer Unitary

Let

$$U_i(\beta_p) = e^{-i \frac{\beta_p}{2} \sum_{x=1}^n \sigma_x^i}.$$

Since Pauli operators acting on different qubits commute, i.e.

$$[\sigma_x^i, \sigma_x^j] = 0, \quad i \neq j,$$

we may split the exponential of the sum into a product:

$$U_i(\beta_p) = \prod_{x=1}^n e^{-i \frac{\beta_p}{2} \sigma_x^i}.$$

Using the Taylor expansion of the exponential,

$$e^{-i\frac{\beta_p}{2}\sigma_x^i} = \sum_{k=0}^{\infty} \frac{\left(-i\frac{\beta_p}{2}\sigma_x^i\right)^k}{k!}.$$

Separating even and odd powers gives

$$e^{-i\frac{\beta_p}{2}\sigma_x^i} = \sum_{k=0}^{\infty} (-1)^k \frac{(\beta_p/2)^{2k}}{(2k)!} I - i \sum_{k=0}^{\infty} (-1)^k \frac{(\beta_p/2)^{2k+1}}{(2k+1)!} \sigma_x^i,$$

since

$$(\sigma_x^i)^{2k} = I, \quad (\sigma_x^i)^{2k+1} = \sigma_x^i.$$

By Euler's formula,

$$e^{-i\frac{\beta_p}{2}\sigma_x^i} = \cos(\beta_p/2)I - i \sin(\beta_p/2)\sigma_x^i.$$

This is exactly the single-qubit rotation gate

$$R_x^i(\beta_p) = e^{-i\frac{\beta_p}{2}\sigma_x^i}.$$

Therefore,

$$U_i(\beta_p) = \prod_{i=1}^n R_x^i(\beta_p).$$

Hence, the mixer unitary decomposes into a product of R_x gates applied to every qubit.

3. Derivation of Decomposition of the Cost Unitary

Let the cost Hamiltonian be

$$H_C = \sum_{i=1}^n \sum_{j=1}^{i-1} \frac{a_{ij}^p}{2} \sigma_z^i \sigma_z^j + \sum_{i=1}^n \frac{b_i^p}{2} \sigma_z^i.$$

Then the cost unitary is

$$U_C(\gamma_p) = e^{-i\gamma_p H_C}.$$

Since all terms consist only of Pauli-Z operators, they commute. Therefore,

$$U_C(\gamma_p) = \prod_{i=1}^n \prod_{j=1}^{i-1} \exp\left(-i\frac{a_{ij}^p}{2} \sigma_z^i \sigma_z^j\right) \prod_{i=1}^n \exp\left(-i\frac{b_i^p}{2} \sigma_z^i\right).$$

Define

$$a_{ij}^p = \gamma_p h_{ij}, \quad b_i^p = \gamma_p h_i.$$

The single-qubit terms become

$$\exp\left(-i\frac{b_i^p}{2} \sigma_z^i\right) = R_z^i(b_i^p).$$

Now consider the two-qubit term

$$\exp\left(-i\frac{a_{ij}^p}{2} \sigma_z^i \sigma_z^j\right).$$

Since

$$\sigma_z^i \sigma_z^j = \sigma_z \otimes \sigma_z = \text{diag}(1, -1, -1, 1),$$

we get

$$\exp\left(-i\frac{a_{ij}^p}{2} \sigma_z^i \sigma_z^j\right) = \text{diag}\left(e^{-ia_{ij}^p/2}, e^{ia_{ij}^p/2}, e^{ia_{ij}^p/2}, e^{-ia_{ij}^p/2}\right).$$

This operator can be implemented using two CNOT gates and one R_z gate:

$$\exp\left(-i\frac{a_{ij}^p}{2} \sigma_z^i \sigma_z^j\right) = \text{CNOT}(i, j) (I \otimes R_z^j(a_{ij}^p)) \text{CNOT}(i, j).$$

Therefore, the full cost unitary decomposes as

$$U_C(\gamma_p) = \prod_{i=1}^n \prod_{j=1}^{i-1} \text{CNOT}(i, j) (I \otimes R_z^j(a_{ij}^p)) \text{CNOT}(i, j) \prod_{i=1}^n R_z^i(b_i^p).$$

Hence, the cost unitary decomposes into two-qubit ZZ rotations implemented with CNOT- R_z -CNOT blocks, together with single-qubit R_z rotations.